Instructions

Repository: https://github.com/SambhavMayani/ADSBDB
Authors: Mario Rodrigo, Sambhav and Wassim
Environment: 64-bit Python 3.11; Windows Git Bash or Linux; Docker for storage. Grant the supervisor repository access if it is private.

Install and run the independent checks

```
git clone https://github.com/SambhavMayani/ADSBDB.git
cd ADSBDB
python -m venv .venv
source .venv/Scripts/activate
python -m pip install -r requirements.txt
python -m pip check
python -m ruff check .
python -m ruff format --check .
python -m pytest -q
python -m orchestration.pipeline --plan
```

On Linux, activate with `source .venv/bin/activate`. Install the pinned PyTorch version from the official CPU wheel index before the same requirements file. Installation needs Internet; default tests need neither model weights nor the wildlife dataset. Expected: passing checks and a 16-step plan.

Exercise real storage and the complete CLI

```
docker compose -p ecosentinel-tests -f docker-compose.test.yml up -d
python -m pytest -m integration -q
python -m exploitation_zone.prepare_models --download
python -m orchestration.smoke --run --workdir data/assessor-smoke
python -m orchestration.smoke --run --workdir data/assessor-smoke
```

Use a new smoke directory initially and the same directory on repetition. The test endpoint is port 19000, without a production volume. Expect three synthetic assets, seven vectors, and zero new vectors on the second run. Synthetic noise/tone inputs test execution, not wildlife relevance.

Run the wildlife interface

The Git repository excludes data, credentials and weights. For the reference demonstration, obtain the authors' data handoff and configure local .env credentials. Alternatively acquire a new sample as documented in docs/user_guide.md; its counts may differ. Prepare BLIP and Qwen explicitly using that guide. Do not regenerate the existing reference index.

```
docker compose up -d
python -m orchestration.reconcile
# Once, with writers stopped and no existing destination:
python -m orchestration.audit --source data/chromadb --snapshot data/search_index
python -m webapp.server
```

Open **http://127.0.0.1:8765**. The three tabs expose text exploration, image/audio similarity and RAG. Try a lynx habitat question, then an animal image plus habitat and diet; inspect citations and diagnostics. Qwen requires at least 2.5 GiB free RAM and can take minutes on CPU. Ctrl+C stops the web server. Never remove the reference Docker volume. See docs/user_guide.md for expected cases and troubleshooting.

EcoSentinel

Multimodal wildlife data management and evidence based retrieval

ADSDB P1
Universitat Politècnica de Catalunya

Mario Rodrigo
Sambhav
Wassim

24 October 2026

Project overview

We built a local pipeline that makes heterogeneous wildlife material searchable and auditable. Images, recordings and text pass through separate ingestion, formatting, quality and vector-indexing stages. A single interface supports related-asset discovery, cross-modal exploration and questions answered from retrieved evidence.

Our engineering priorities are reproducibility, explicit contracts, incremental execution and recoverable failures. The reference corpus contains 198 accepted assets and 6,021 persisted embeddings. We preserve original media and distinguish technical data quality, retrieval relevance and biological correctness.

This report explains the domain, the value of the system, the reasons for each zone and model, the three analytical Tasks and the operations environment. It records observed validation and limitations rather than treating plausible model output as verified identification.

Context and data 2 | Data management backbone 3–5
Multi-modal Tasks 6–7 | Operations and validation 8

Context and Data

Domain and purpose

Wildlife information is fragmented across photographs, field recordings and species descriptions. A researcher who finds a relevant photograph may still have to search another catalogue for vocalisations and another source for habitat information. We chose Iberian fauna to make that cross-modal discovery useful within a clearly bounded domain.

EcoSentinel is a prototype evidence workspace for conservation educators, field-study teams and biodiversity analysts. A user can start from an image, recording or phrase, inspect related assets and ask a question grounded in the collection. The potential value is less manual catalogue switching and a traceable route from an answer to its underlying evidence. These are proposed benefits; we have not measured business adoption or time savings.

Sources and corpus

Modality	Reference assets	Source and acquisition
Images	100	Wikimedia Commons API; ten images per species
Audio	34	Xeno-canto API; recordings available for nine species
Text	64	20 Wikipedia articles and 44 GBIF description records

The ten species include Iberian lynx, wolf, deer, wild boar, eagle, eagle owl, kingfisher, robin, Perez's frog and ocellated lizard. Scientific identifiers in config/species.yaml keep the zones consistent. We deliberately include audio as a third modality: it adds ecological context and requires distinct decoding, quality metrics and an aligned audio-text encoder.

Acquisition scripts save source URLs, provider identifiers, hashes and available attribution/licence fields beside the raw files. Wikipedia records also retain revision information. Downloads are optional; saved raw data can enter the same pipeline offline. We store provider labels as provenance, not expert-verified species annotations.

Representativeness and responsible reuse

The sample is small and curated. Images are balanced by design, but text coverage ranges from two to twelve documents per species; audio is absent for the lizard and limited to two lynx recordings. Source availability therefore influences retrieval. A technically valid record is not automatically a representative ecological observation.

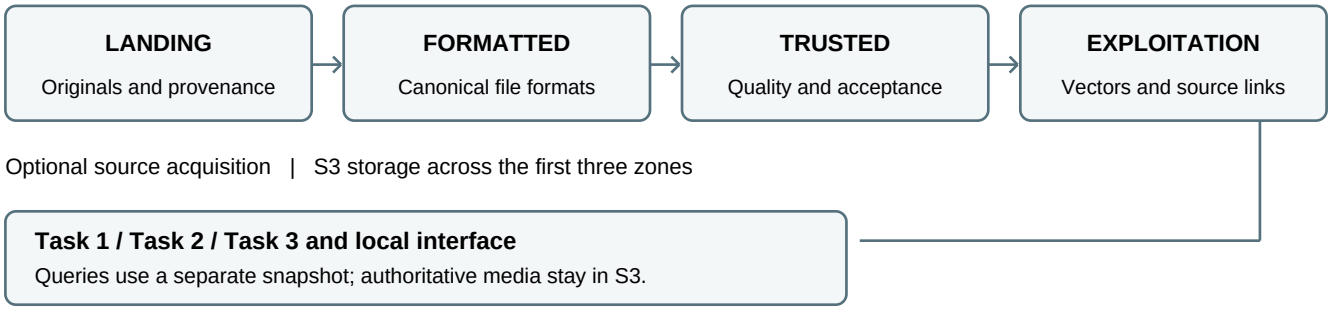
Raw assets and weights are not bundled in Git. Existing metadata includes different reuse terms, including non-commercial audio licences. Attribution must travel with a dataset handoff. The academic prototype does not establish commercial redistribution rights; a production application would need an asset-by-asset permissions review and expert validation.

Source metadata and model references: docs/data_sources.md. Aggregate corpus evidence: docs/evidence/quality_*.json. The recorded counts refer to the audited reference installation, not a promise about later API downloads.

The Data Management Backbone

Independent zones with explicit responsibilities

We separate reusable data management from model-specific preparation. Python packages match the four zones; Landing, Formatted and Trusted use separate S3 buckets, while Exploitation uses embedded Chroma. The Docker service is the pinned SILO S3-compatible distribution. Using S3 object storage fits heterogeneous media without forcing binary payloads into relational rows; Chroma then provides vector-oriented retrieval without another server.



Landing preserves evidence

Optional source adapters first create raw files and metadata sidecars. Before ingestion, a Security Gate checks extensions, signatures, filename/size limits, image structure and archive expansion limits. It is a processing safeguard, not a malware scanner. Temporal Landing receives accepted inputs; Persistent Landing organises originals by modality and species with traceable naming and ingestion metadata.

Content identities and completion markers prevent duplicate promotion. Copy verification checks provenance, size and SHA-256 before completion. We retain Temporal objects by default; an explicit cleanup option is available only after verifying the persistent copy. Retention costs space but makes recovery safer. A failed copy or incomplete marker cannot be interpreted as a completed asset.

Formatted standardises syntax

Type	Canonical representation	Why and trade-off
Image	JPEG RGB	Wide decoder support and manageable storage; JPEG is lossy, so the original remains in Persistent.
Audio	FLAC PCM16	Portable lossless compression of the selected PCM representation; original higher precision is retained upstream.
Text	UTF-8, NFC and LF	Consistent decoding and paragraph handling across HTML, JSON, documents and plain text.

One converter per modality makes failures and tests independent. The stage avoids model-specific cropping, tokenisation or resampling. Audio keeps its source rate/channels when converted; segmentation and 48 kHz mono preparation belong downstream. Unsupported or unextractable content produces an explicit failure instead of a fabricated empty record.

Separate objects cost storage, but preserve a reversible lineage from each derived asset to its original. Configuration and checksums record the transformation boundary; a distributed transaction across all stores is not assumed.

Trusted Zone and Quality Analysis

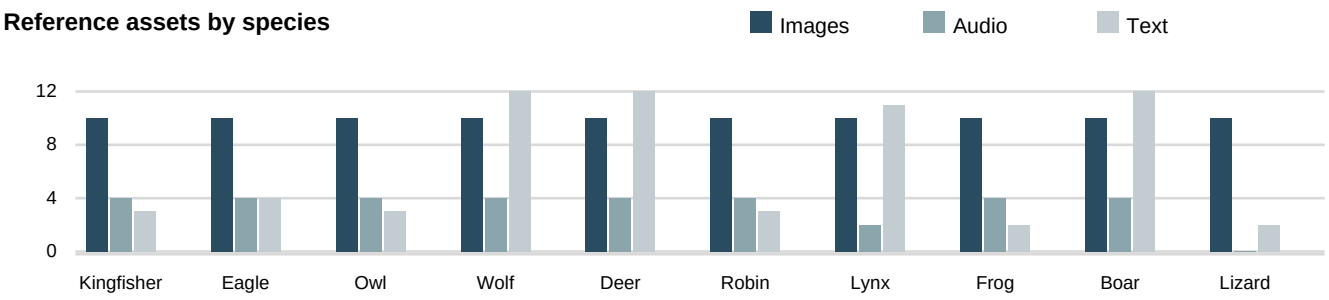
Trusted evaluates generic usability and lineage before any particular model is selected. YAML policies keep thresholds visible and versioned. Each modality has its own validator and measurements, with shared publication utilities. Accepted objects and JSON quality manifests feed reproducible HTML, CSV and plot reports.

Conservative decisions preserve ecological signal

Images must decode and satisfy technical dimensions and provenance. Exact hashes and pHash candidates followed by SSIM/colour confirmation detect duplicates. Darkness, low sharpness, monochrome appearance and possible watermarks are primarily review signals. Audio checks integrity, duration, amplitude statistics and exact PCM duplicates; it does not denoise, trim silence, normalise loudness or discard distant calls. Text preserves language and content, with strict decoding, exact duplicate checks, near-duplicate warnings and privacy-pattern counts.

These choices protect future analytical uses. A quiet interval may be ecologically meaningful; a short GBIF description need not resemble a full article. We reject technical failures and qualifying duplicates, distinguish operational errors from quality rejection, and retain previously accepted bytes when a later evaluation suggests retirement. Reconciliation exposes obsolete index references instead of deleting them silently.

Observed reference quality



Modality	Observed quality indicators	Interpretation
Images	100/100 accepted; 14 monochrome-like, 32 possible-watermark flags	Flags may overlap; they are not verified defects.
Audio	34/34 accepted; 2 low-RMS and 9 low-energy flags; median 40.91 s	Quiet recordings retained; longest recording 752.37 s.
Text	64/64 accepted; median 208.5 words; range 33–13,869	Different source lengths motivate later segmentation.

The 19 October reference reports record 100% provenance-field coverage and no removals for this curated batch. This means the measured fields were populated, not that all content was correct. Before/after species distributions are unchanged; the lack of lizard audio remains visible. Synthetic pipeline reports are kept separate from these corpus observations.

Limitations are explicit: watermark and near-duplicate heuristics have no labelled precision/recall benchmark; text near-duplicate comparison is quadratic at the current small scale. Future expansion needs indexed duplicate candidates and annotated evaluation. Evidence: docs/evidence/quality_images.json, quality_audio.json and quality_text.json.

Exploitation Zone and Model Choices

We store embeddings with asset IDs, segment positions, source keys, content hashes and policy fingerprints. The authoritative media remain in S3. Five collections meet the single-modality requirement while adding two explicitly shared spaces; a vector from one model is never treated as compatible with another merely because its dimension matches.

Collection	Encoder and dimension	Vectors
Images	CLIP ViT-B/32, 512	100
Audio	CLAP HTSAT unfused, 512	357
Text	Multilingual E5 small, 384	547
Image and text	CLIP image + aligned multilingual text, 512	1,763
Audio and text	CLAP audio/text towers, 512	3,254

Why these pretrained encoders

CLIP supplies a common image/text representation, enabling semantic visual discovery without training on our small corpus. The multilingual text adapter is aligned to CLIP ViT-B/32, so Spanish/English queries can share the image space. Its captions-oriented representation is useful for visual concepts but less suited to detailed biological prose.

E5 small provides multilingual passage retrieval with a smaller CPU footprint than larger alternatives. We use the prescribed query and passage prefixes and 384-dimensional normalised vectors. A dedicated text encoder complements CLIP for habitat and behavioural descriptions. **CLAP** adds genuine audio/text alignment; a speech recogniser would lose non-verbal animal sounds. The unfused model is used with 10-second, 48 kHz mono windows. Its language/domain limitations make English acoustic prompts preferable.

Segmentation and compatible re-execution

Text windows use E5 320/40, multilingual CLIP 96/12 and CLAP 60/8 token window/overlap settings. These respect each configured encoder budget and reduce boundary loss. All tail segments are covered. Overlap and multiple spaces explain why 198 assets produce 6,021 vectors; the vectors are not 6,021 independent observations.

Stable record IDs and complete part coverage allow unchanged assets to skip inference. Source or policy changes require explicit compatible updates; incompatible collections fail rather than mixing representations. Interrupted publication can be retried. Historical v1 fingerprints are preserved, while an immutable model manifest resolves previously null revisions. That manifest records observed cache commits; it is not retrospective proof of every original inference.

Models are loaded sequentially with small batches and CPU as the default. This bounds live model families, although full-file audio decoding still scales with duration. Chroma snapshots protect the original index during queries; freshness is an explicit operational responsibility.

Model cards and exact revisions: https://huggingface.co/sentence-transformers/clip-ViT-B-32-multilingual-v1

Additional primary references: huggingface.co/intfloat/multilingual-e5-small and huggingface.co/laion/clap-htsat-unfused. Exact commits and policies are in config/model_manifest.json and config/exploitation.yaml.

Multi Modal Tasks and User Interface

Task 1 discovers related assets within a modality

A reference image retrieves images; a recording retrieves recordings; a text query uses E5 for texts. Stored-record queries reuse the existing vector and exclude the entire reference asset, including its other segments. An external query is embedded in memory without ingestion. Results are grouped by asset so a long document or recording does not occupy the ranking solely through repeated chunks.

We use cosine similarity within each compatible space. For at most 64 candidates, a bounded exact scan handles small collections; larger sets use Chroma HNSW. Tests cover both paths. The resulting scores indicate geometric similarity, not calibrated species probabilities. Some visually related but taxonomically different animals can rank together.

Task 2 connects modalities through aligned spaces

The image-text and audio-text collections each contain both modalities. Text can retrieve photographs and descriptions, or recordings and descriptions. Image/audio inputs can also retrieve mixed results. An explicit diversity policy allocates available results across modalities, making the shared information visible instead of allowing numerous text chunks to dominate.

For a federated query, E5, CLIP and CLAP produce separate rankings. Reciprocal Rank Fusion combines asset ranks using a constant of 60. We chose rank fusion because cosine magnitudes from unrelated encoders are not comparable. Fusion and modality quotas change the ranking objective: the output is a useful mixed view, not a pure global cosine top-k. Its value should eventually be assessed with relevance labels.

One local interface exposes the complete workflow

Tab	Task and user workflow
Explorar por texto	Text-only E5, either shared space, or all modalities; inspect sources and filters.
Buscar similares	Upload an image or short recording; choose same-modality or mixed retrieval.
Preguntar al corpus · RAG	Ask about the corpus, optionally with an uploaded or stored image; inspect cited claims and diagnostics.

Result cards expose source lineage and related-asset actions. RAG shows candidate species as hypotheses, links claims to reference excerpts, and distinguishes complete answers, partial answers, abstention and technical errors. The user can inspect the model revision, request ID and runtime without needing a terminal.

The web server binds to loopback, validates host/origin and upload limits, and serialises CPU work. The static frontend requires no build toolchain. This reduces setup complexity for assessment, but provides no public authentication, multiuser capacity or deployment-grade isolation. A hosted service would need a separate architecture and operational controls.

A concise case table and expected behaviors are in docs/user_guide.md. Automated HTTP tests cover real request/response handling, input rejection, provenance and protected storage boundaries. Native-model acceptance complements those structural tests.

Task 3 Retrieval Augmented Generation

Task 3 automatically retrieves stored evidence before invoking a pretrained generator. We combine BLIP image captioning, CLIP visual similarity and E5 text retrieval with Qwen2.5-3B-Instruct-GGUF in Q4_K_M form. This implements the assignment's generative pipeline requirement and its recommendation to reuse Tasks 1 and 2.

A bounded multimodal retrieval pipeline

For image input, BLIP produces an unverified description and CLIP retrieves visual candidates. Candidate labels guide text searches by requested aspect, such as habitat or diet; a bounded lexical scan complements E5 and retains distinct passages from the same article. A global query remains available so retrieval is not reduced to a hard species assignment. We avoid injecting a noisy caption verbatim into every biological search.

Rank fusion selects references, then an evidence selector packs at most eight literal units and 1,600 characters for generation. Full reference excerpts remain available in the interface. The prompt treats retrieved material as data, preserves uncertainty about the uploaded animal and asks for concise claims backed by allowed evidence IDs.

Why Qwen and BLIP

A 3B quantised instruction model makes local CPU execution practical without a paid API or a GPU requirement. The pinned GGUF is verified by SHA-256; llama.cpp generates under a JSON grammar. BLIP provides a separate, auditable image-to-text stage. We accepted the information loss of this adapter to control memory and keep retrieval and generation independently testable. Qwen receives text, **not image pixels**; this is not a native visual question-answering model like the assignment's LLaVA example.

Multi-aspect requests use constrained slots for up to two claims. An unsupported aspect can return null and becomes an explicit partial answer. Post-validation checks structure, allowed source IDs, exact quote membership, length and obvious incomplete clauses. It does not prove semantic entailment. Domain and aspect planning use vocabulary heuristics and can miss unfamiliar phrasing.

Failure states and observed behavior

An evidence-based abstention differs from invalid JSON, a token limit or a generation timeout. Technical failures keep references/candidates visible. Generation is limited to 320 output tokens and 120 seconds, with four CPU threads and a 2.5 GiB available-memory check. All configured revisions, policy hashes and stage timings are auditable; uploads are not added to the corpus.

Native acceptance recorded five CLI cases: text habitat, stored-image habitat, conditional frog identification, unsupported genome and an unrelated query. A separate browser run answered both frog habitat and diet with verified excerpts in 131.8 seconds total (76.1 seconds generation). One lynx response was less directly relevant. These checks demonstrate execution and failure handling, not broad species-identification accuracy.

The reference model card identifies the Qwen Research licence; deployment beyond academic assessment needs review of its terms. BLIP uses the model card linked below. Native outputs and preservation summaries are recorded in docs/evidence/native_models.json.

Qwen GGUF: https://huggingface.co/Qwen/Qwen2.5-3B-Instruct-GGUF

BLIP captioning: https://huggingface.co/Salesforce/blip-image-captioning-base

Operations and Validation

From independent scripts to a recoverable pipeline

One orchestrator runs 16 sequential steps: storage checks, ingestion/promotion, three formatters, three validators, five collection stages and final reconciliation. Optional acquisition adds three steps. Subprocesses use the active Python interpreter without a shell. A local OS lock prevents overlapping orchestrator runs in the same checkout; it does not coordinate manual writers or another machine.

Each stage has a deadline, UTF-8 logs and an atomic JSON audit trail. A failure stops later stages; interrupted children are terminated. Restarting at a chosen stage rechecks persisted objects and vector completeness instead of trusting the previous run report. Shared S3 clients apply bounded timeouts/retries. Credentials stay in .env and configured secrets are redacted from pipeline logs.

Evidence at different levels

Level	Observed result and scope
Static quality	Ruff checks and one formatting standard across application code and tests; one pinned requirements file.
Offline suite	344 passing tests; 63.82% statement coverage. Generated fixtures, fault injection, temporary Chroma and HTTP.
Storage integration	One passing end-to-end test using isolated real S3/Chroma and deterministic encoders; repeat execution checked.
CPU acceptance	Full CLI with three synthetic assets and seven vectors; native retrieval and RAG checked separately on the reference corpus.
Preservation	Original Chroma file hashes and 6,021 counts checked; S3 keys, sizes and ETags compared. No corpus regeneration.

The final review strengthened segment completeness, rejected non-finite limits, unified S3 clients and removed obsolete patch redirects. Small persistent test indexes are explicitly materialised and closed on Windows. Regression tests exercise these failure boundaries.

Continuous integration and reproducibility limits

GitHub Actions installs the same dependencies, runs offline checks on Windows and Linux, enforces a 60% coverage floor and runs isolated S3 integration on Linux. JUnit and coverage artifacts are retained. A locally passing suite does not prove a remote job passed; the authors must publish the final commit and verify that workflow before submission. Native model inference remains opt-in because caches, RAM and timings depend on the host.

The code and synthetic demonstration are reproducible from a clone. Exact corpus reproduction additionally needs its data handoff and pinned model cache. A fresh download may change source content. Other limitations include snapshot freshness, lack of a distributed transaction, quadratic text duplicate comparisons, duration-dependent audio memory, and the absence of a labelled retrieval/biological benchmark.

Future evaluation needs held-out external inputs, absent species, relevance labels and false-abstention analysis.

Assessment references: official ADSDB P1 guidelines, constraints C1–C16 and delivery section; mapping in docs/p1_requirements.md. Operational and test details: docs/pipeline_orchestration.md, docs/testing_and_validation.md and docs/evidence/.